Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни «Бази даних»

Виконав:
Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірив:

Хмельницький Арсеній Андрійович

Київ 2026

Короткий виклад вимог

Завдання:

Використовуючи базу даних VideoHub з Lab 2, виконати аналітичні запити (OLAP — Online Analytical Processing): агрегацію даних, групування, фільтрування груп та з'єднання таблиць.

Чим OLAP відрізняється від OLTP (Lab 3):

У Lab 3 кожна операція працювала з одним рядком (додати користувача, змінити опис) — це транзакційне навантаження. Тут навпаки: запит обробляє **багато рядків і повертає одну підсумкову відповідь** — «скільки всього відео», «сумарні перегляди автора». Це аналітичне навантаження.

Інструменти лабораторної:

- **Агрегатні функції** — COUNT, SUM, MIN, MAX, AVG: стискають багато рядків в одне значення
- **GROUP BY** — розбиває рядки на групи за значенням колонки
- **HAVING** — фільтрує ГРУПИ після агрегації (WHERE фільтрує рядки ДО)
- **JOIN** — з'єднує таблиці: INNER, LEFT, FULL OUTER, CROSS
- **Підзапити** — запит всередині запиту

Базова агрегація

Усі п'ять агрегатів виконані **одним запитом** — один прохід по таблиці замість п'яти окремих:

```
SELECT COUNT(*)                 AS total_videos,
       COUNT(DISTINCT author_id) AS unique_authors,
       SUM(views)                AS total_views,
       MIN(views)                AS min_views,
       MAX(views)                AS max_views,
       ROUND(AVG(views), 0)      AS avg_views
FROM "Video";
```

Пояснення:

- `COUNT(*)` — рахує всі рядки таблиці (7 відео)
- `COUNT(DISTINCT author_id)` — рахує унікальні значення: 5 авторів (у denys_dev 3 відео, тому авторів менше, ніж відео)

- `SUM(views)` — сумарні перегляди всіх відео (186 300)
- `MIN` / `MAX` — найменш і найбільш популярне відео (5 230 та 89 300)
- `AVG(views)` — середнє; `ROUND(..., 0)` округлює до цілого (26 614), бо AVG повертає дробове значення

Групування (GROUP BY)

1. Статистика кожного автора — кількість відео та сумарні перегляди одним запитом:

```
SELECT u.username,
       COUNT(v.id)  AS video_count,
       SUM(v.views) AS total_views
FROM "User" u
INNER JOIN "Video" v ON u.id = v.author_id
GROUP BY u.username
ORDER BY total_views DESC;
```

`INNER JOIN` з'єднує користувачів з їхніми відео, `GROUP BY u.username` збирає відео одного автора в групу, агрегати рахують статистику групи. `ORDER BY total_views DESC` — найпопулярніші автори зверху. Колонка `username` унікальна (UNIQUE constraint), тому групування за нею коректне.

2. Кількість коментарів для кожного відео:

```
SELECT v.title, COUNT(c.id) AS comment_count
FROM "Video" v
LEFT JOIN "Comment" c ON v.id = c.video_id
GROUP BY v.id, v.title
ORDER BY comment_count DESC;
```

Тут навмисно **LEFT JOIN**, а не INNER: відео без жодного коментаря теж потрапляють у список з нульовим рахунком. INNER JOIN викинув би їх — і аналітика була б неповною. `GROUP BY v.id, v.title` (а не тільки title) — захист від склеювання відео з однаковими назвами: ідентифікатор гарантує, що кожне відео — окрема група.

Фільтрування груп (HAVING)

1. Автори з більш ніж 1 відео:

```
SELECT u.username, COUNT(v.id) AS video_count
FROM "User" u
INNER JOIN "Video" v ON u.id = v.author_id
GROUP BY u.username
HAVING COUNT(v.id) > 1;
```

2. Відео з більш ніж 2 коментарями:

```
SELECT v.title, COUNT(c.id) AS comment_count
FROM "Video" v
INNER JOIN "Comment" c ON v.id = c.video_id
```

```
GROUP BY v.id, v.title
HAVING COUNT(c.id) > 2;
```

3. Автори з сумарними переглядами > 20000:

```
SELECT u.username, SUM(v.views) AS total_views
FROM "User" u
INNER JOIN "Video" v ON u.id = v.author_id
GROUP BY u.username
HAVING SUM(v.views) > 20000;
```

Чому HAVING, а не WHERE:

Логічний порядок виконання запиту: `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`. На етапі WHERE груп ще не існує — агрегат `COUNT(v.id)` там неможливий синтаксично. HAVING виконується ПІСЛЯ групування і бачить готові групи з їхніми агрегатами. WHERE фільтрує рядки, HAVING — групи.

JOIN (різні типи з'єднань)

1. INNER JOIN — відео з іменами авторів (тільки збіги):

```
SELECT v.title, u.username AS author
FROM "Video" v
INNER JOIN "User" u ON v.author_id = u.id;
```

Повертає лише пари, де збіг є в обох таблицях.

2. LEFT JOIN — усі користувачі, навіть без відео:

```
SELECT u.username, v.title
FROM "User" u
LEFT JOIN "Video" v ON v.author_id = u.id;
```

Усі рядки лівої таблиці зберігаються; де відео нема — права сторона NULL. Користувач test_guest ще нічого не завантажив — і він у результаті з NULL.

Анти-джойн — користувачі БЕЗ жодного відео:

```
SELECT u.username
FROM "User" u
LEFT JOIN "Video" v ON v.author_id = u.id
WHERE v.id IS NULL;
```

Класичний патерн «знайти тих, у кого нема зв'язку»: LEFT JOIN залишає всіх, а `WHERE v.id IS NULL` відбирає рядки, де збігу не було.

3. FULL OUTER JOIN — усі користувачі та всі відео:

```
SELECT u.username, v.title
FROM "User" u
```

```
FULL OUTER JOIN "Video" v ON v.author_id = u.id;
```

FULL OUTER зберігає рядки обох таблиць навіть без збігу. У нашій схемі відео без автора **неможливе** (`author_id NOT NULL` + FK), тому «правих-без-лівих» не буває і FULL тут дає той самий результат, що LEFT — додаткові NULL-рядки з'являються лише з боку користувачів без відео.

4. CROSS JOIN — всі комбінації користувачів та відео:

```
SELECT u.username AS potential_viewer, v.title
FROM "User" u
CROSS JOIN "Video" v;
```

Декартів добуток: кожен користувач × кожне відео = 6 × 7 = 42 рядки. Практичний сенс — «potential audience»: кому яку рекомендацію можна показати, до застосування фільтрів.

Багатотаблична агрегація

1. Топ-3 автори за сумарними переглядами:

```
SELECT u.username, SUM(v.views) AS total_views
FROM "User" u
INNER JOIN "Video" v ON u.id = v.author_id
GROUP BY u.username
ORDER BY total_views DESC
LIMIT 3;
```

Комбінація JOIN + GROUP BY + ORDER BY + LIMIT: повний рейтинг авторів, обрізаний до трійки лідерів.

2. Кожне відео з кількістю лайків та коментарів:

```
SELECT v.title,
       COUNT(DISTINCT l.user_id) AS like_count,
       COUNT(DISTINCT c.id) AS comment_count
FROM "Video" v
LEFT JOIN "Like" l ON l.video_id = v.id
LEFT JOIN "Comment" c ON c.video_id = v.id
GROUP BY v.id, v.title
ORDER BY like_count DESC, comment_count DESC;
```

Тут важливий нюанс: подвійний LEFT JOIN **розмножує рядки** — для відео з 2 лайками і 3 коментарями утворюється 2 × 3 = 6 проміжних рядків, і звичайний `COUNT(*)` порахував би 6. `COUNT(DISTINCT ...)` рахує унікальні значення і нейтралізує це розмноження — так отримуємо чесні лічильники.

3. Підзапит — автори, чий середній перегляд вищий за загальний:

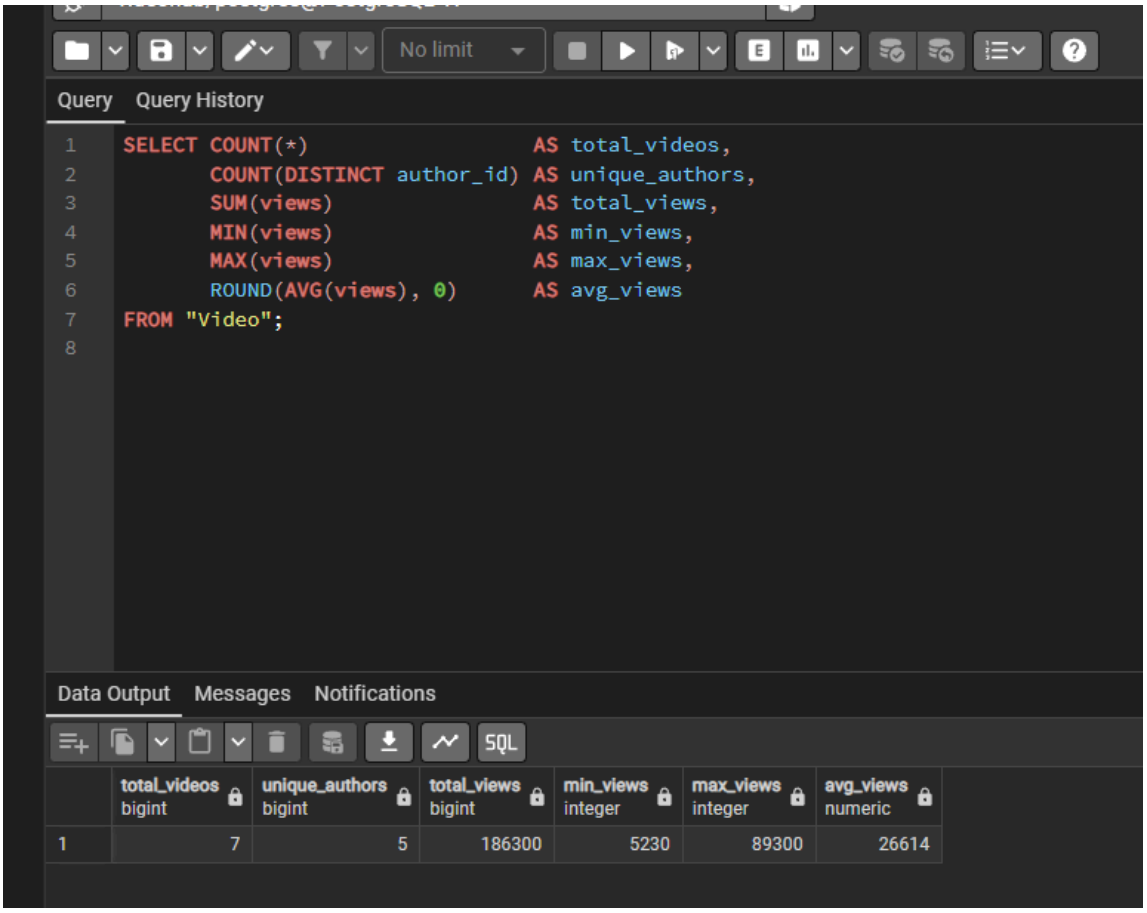
```
SELECT u.username, ROUND(AVG(v.views), 0) AS avg_views
FROM "User" u
INNER JOIN "Video" v ON u.id = v.author_id
```

```
GROUP BY u.username
HAVING AVG(v.views) > (SELECT AVG(views) FROM "Video");
```

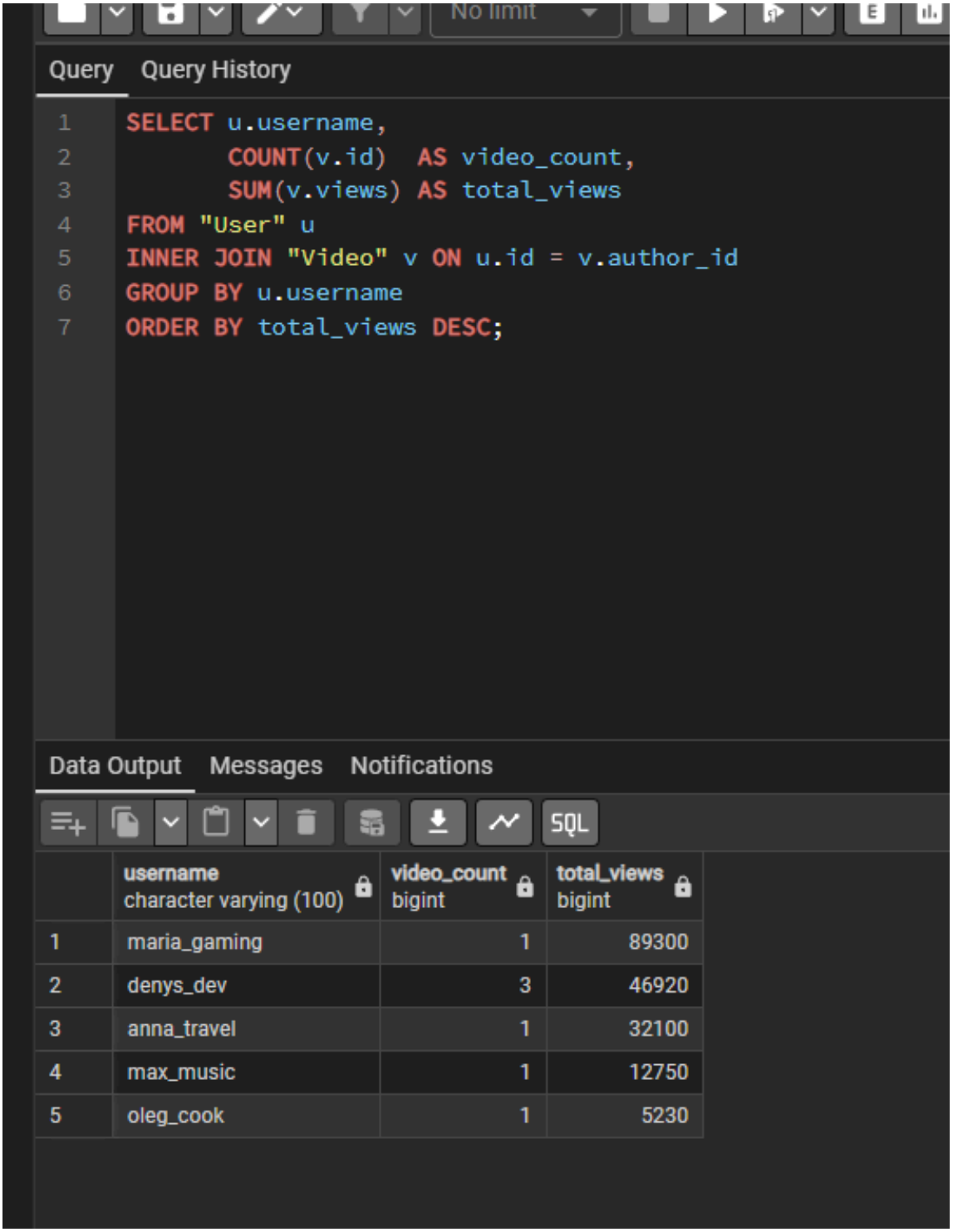
Підзапит `(SELECT AVG(views) FROM "Video")` обчислює загальний середній по платформі один раз; HAVING порівнює середнє кожного автора з цим порогом. У результаті — автори, чиї відео в середньому популярніші за платформу в цілому.

Скріншоти

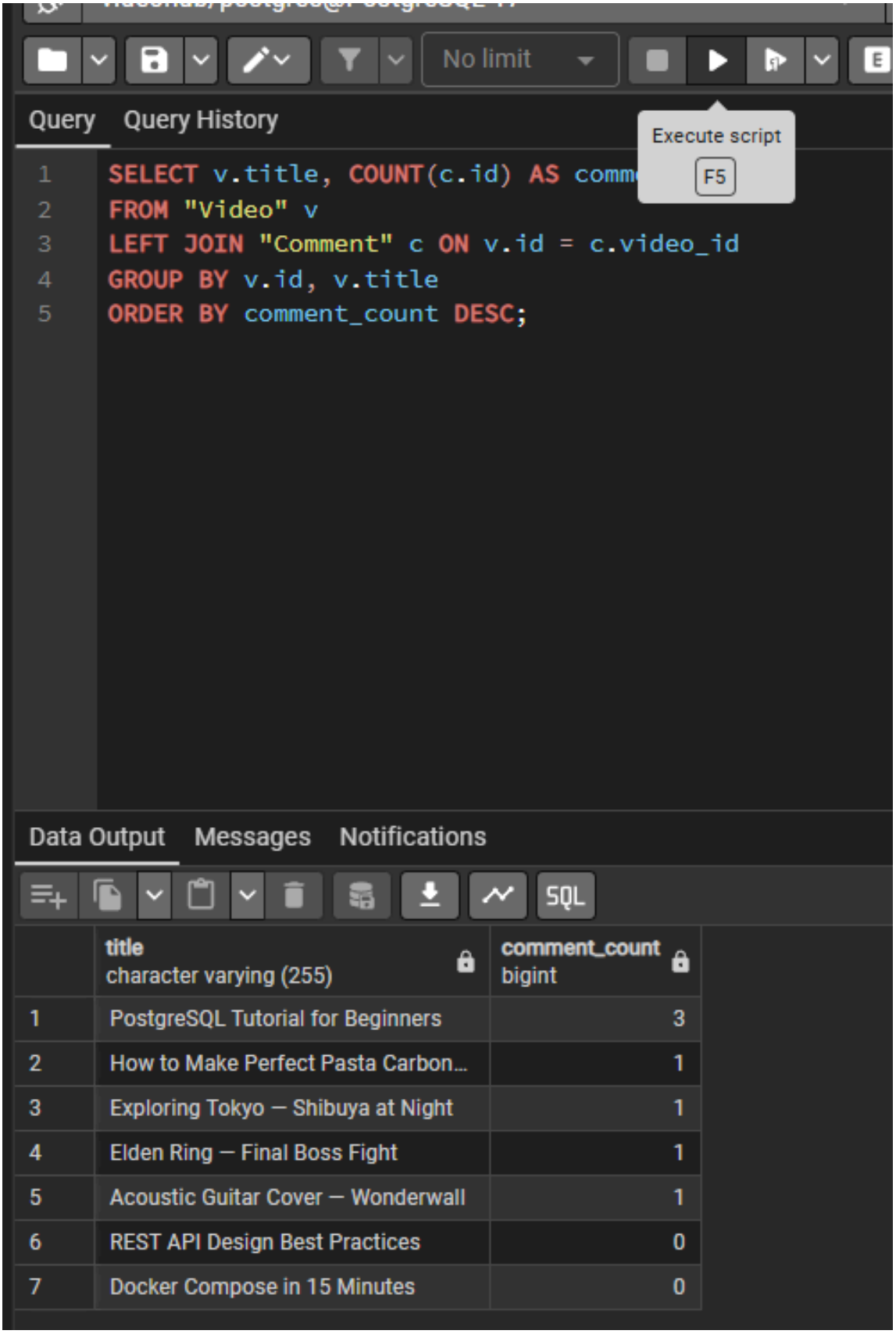
Базова агрегація — всі п'ять функцій одним запитом:



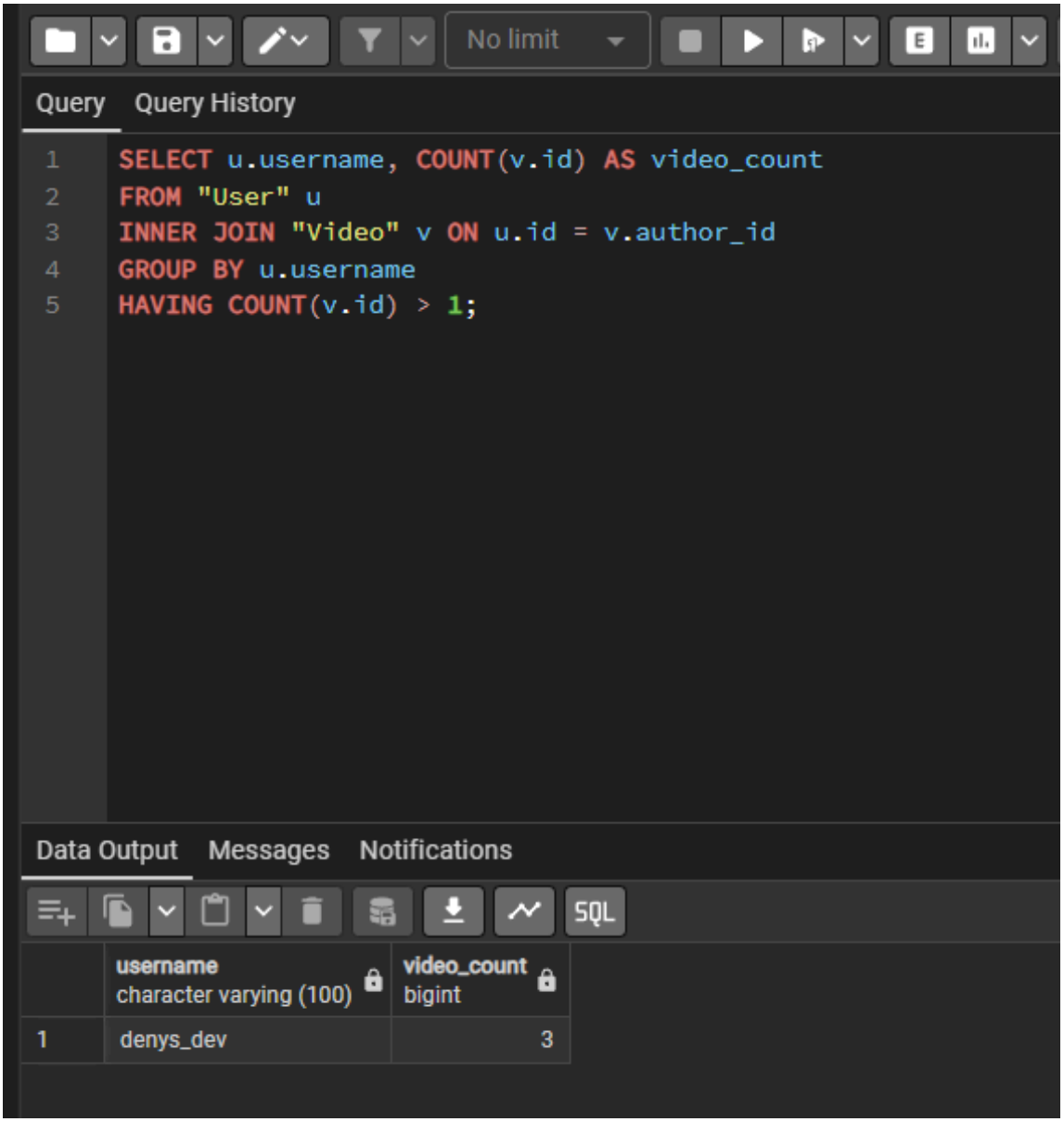
GROUP BY — статистика кожного автора (кількість відео + сумарні перегляди):



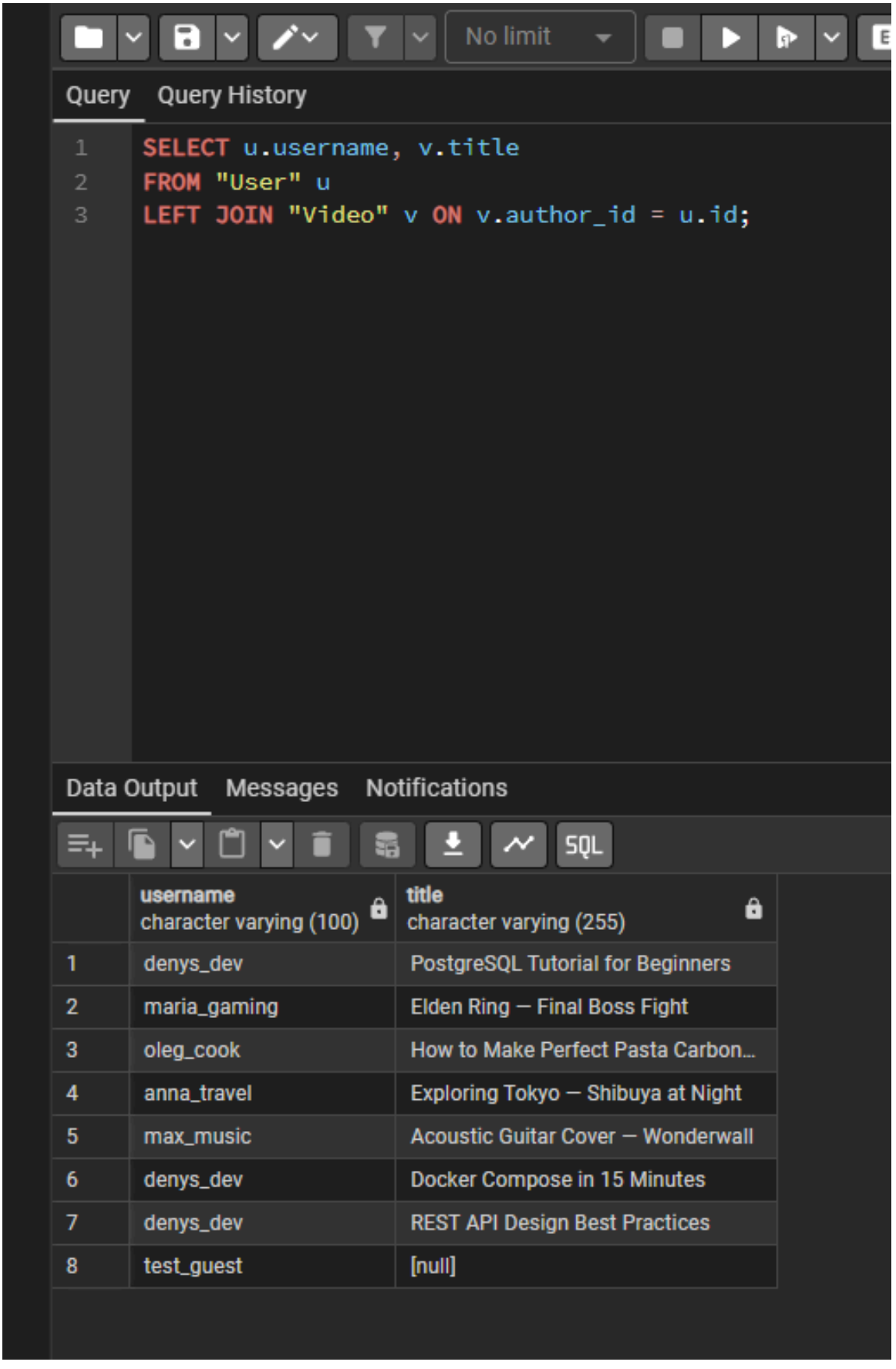
GROUP BY — коментарі кожного відео (LEFT JOIN, відео без коментарів з 0):



HAVING — автори з більш ніж 1 відео:

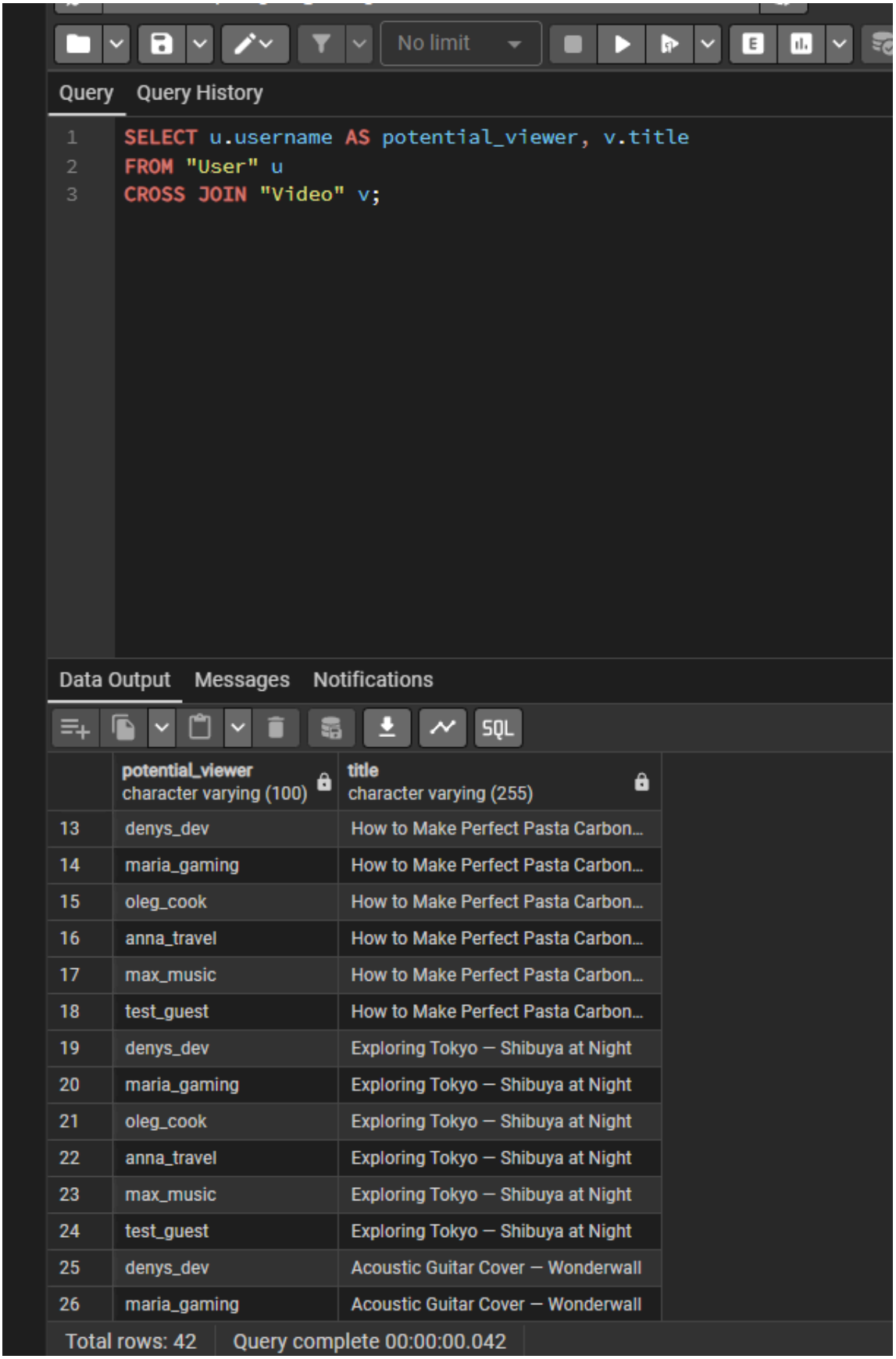


LEFT JOIN — усі користувачі, test_guest без відео:

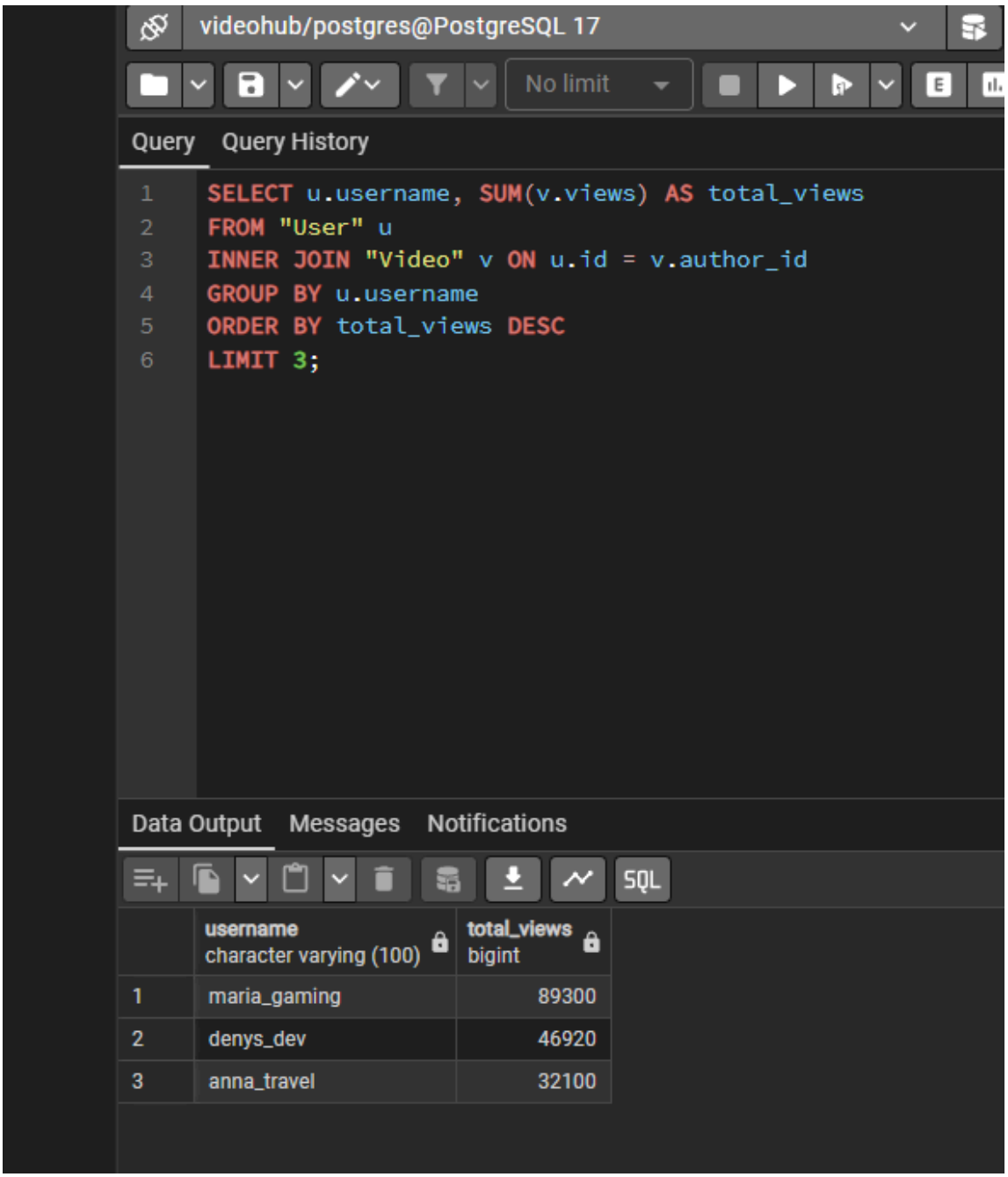


	username character varying (100)	title character varying (255)
1	denys_dev	PostgreSQL Tutorial for Beginners
2	maria_gaming	Elden Ring — Final Boss Fight
3	oleg_cook	How to Make Perfect Pasta Carbon...
4	anna_travel	Exploring Tokyo — Shibuya at Night
5	max_music	Acoustic Guitar Cover — Wonderwall
6	denys_dev	Docker Compose in 15 Minutes
7	denys_dev	REST API Design Best Practices
8	test_guest	[null]

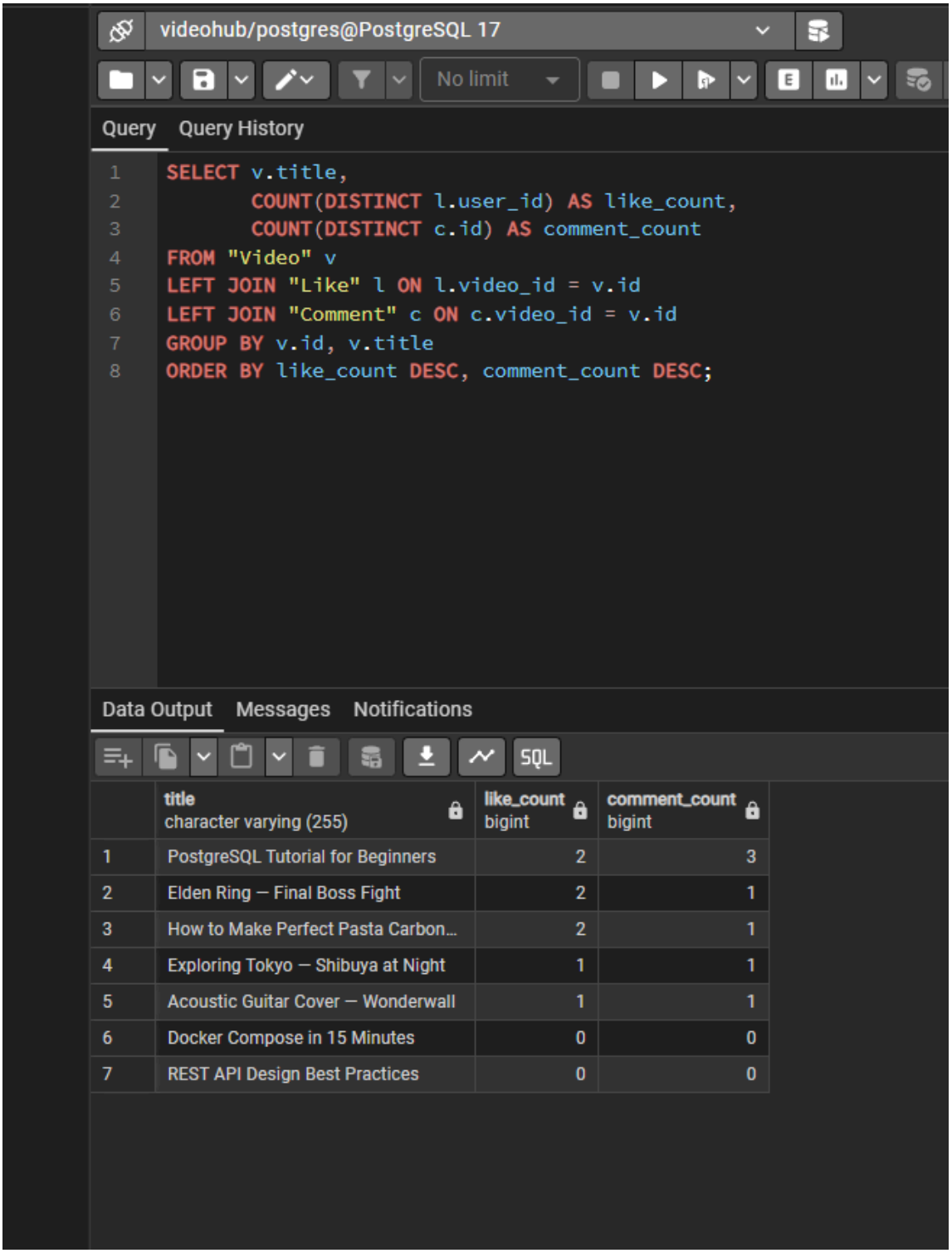
CROSS JOIN — всі комбінації користувачів та відео (42 рядки):


No limit
Query
Query History
1 SELECT u.username AS potential_viewer, v.title
2 FROM "User" u
3 CROSS JOIN "Video" v;
Data Output
Messages
Notifications
SQL
potential_viewer
character varying (100)
title
character varying (255)
13 denys_dev How to Make Perfect Pasta Carbon...
14 maria_gaming How to Make Perfect Pasta Carbon...
15 oleg_cook How to Make Perfect Pasta Carbon...
16 anna_travel How to Make Perfect Pasta Carbon...
17 max_music How to Make Perfect Pasta Carbon...
18 test_guest How to Make Perfect Pasta Carbon...
19 denys_dev Exploring Tokyo — Shibuya at Night
20 maria_gaming Exploring Tokyo — Shibuya at Night
21 oleg_cook Exploring Tokyo — Shibuya at Night
22 anna_travel Exploring Tokyo — Shibuya at Night
23 max_music Exploring Tokyo — Shibuya at Night
24 test_guest Exploring Tokyo — Shibuya at Night
25 denys_dev Acoustic Guitar Cover — Wonderwall
26 maria_gaming Acoustic Guitar Cover — Wonderwall
Total rows: 42
Query complete 00:00:00.042

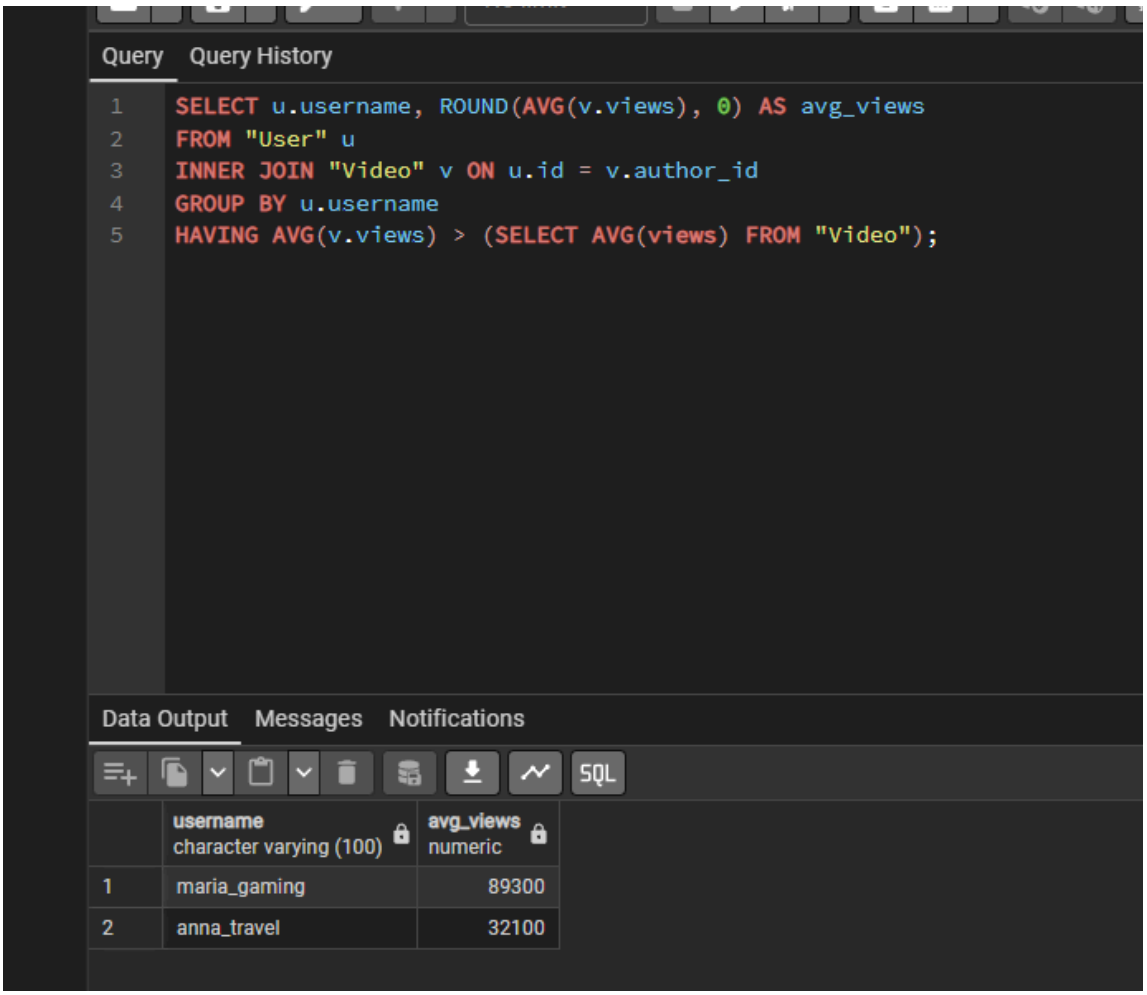
Топ-3 автори за сумарними переглядами:



Кожне відео з кількістю лайків та коментарів (COUNT DISTINCT проти розмноження рядків):



Підзапит — автори з середнім переглядом вище загального:



Будь які припущення чи обмеження

1. Додаткові тестові дані.

Базові дані з Lab 2 рівномірні: у кожного з 5 авторів рівно одне відео, в кожного відео рівно один коментар. За таких даних запити «автори з більш ніж 1 відео» та «відео з більш ніж 2 коментарями» повертали б порожній результат — аналітику нічого показувати. Тому перед виконанням запитів додано нерівномірні дані: другий користувач без відео (test_guest), ще два відео для denys_dev, ще два коментарі до першого відео, ще три лайки. Це відображає реальність: аналітика осмислена лише на даних з природною варіацією.

2. Групування за username замість id.

Колонка `username` має UNIQUE constraint, тому групування за нею коректне — два різні користувачі не можуть мати однаковий username. Загальніший патерн — `GROUP BY u.id, u.username` (за первинним ключем); у запитах з назвами відео (`GROUP BY v.id, v.title`) використано саме його, бо `title` унікальним не є.

3. FULL OUTER JOIN збігається з LEFT JOIN на цій схемі.

Відео без автора неможливе за дизайном (`author_id NOT NULL` + зовнішній ключ), тому «непарні» рядки можливі лише з боку користувачів. FULL OUTER JOIN продемонстровано, але його результат тут ідентичний LEFT JOIN — це слідство цілісності схеми, а не обмеження запиту.

4. COUNT(DISTINCT) у багатотабличній агрегації.

Подвійний LEFT JOIN до Like та Comment розмножує проміжні рядки (кожен лайк поєднується з кожним коментарем того ж відео). Звичайний COUNT дав би завищені числа; COUNT(DISTINCT) рахує унікальних користувачів-лайкачів та унікальні коментарі.